

DOCKER & KUBERNETES

①

CRASH COURSE

1. DOCKER FUNDAMENTALS

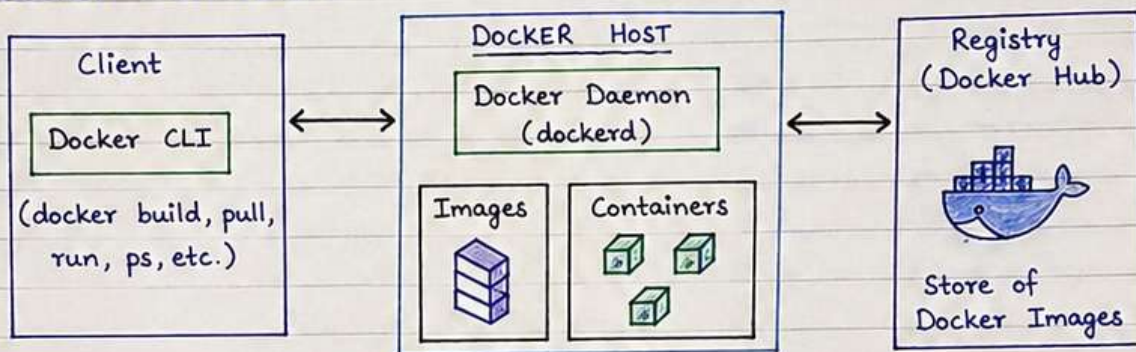
→ What is Docker?

Docker is a platform that helps us to develop, ship and run applications inside containers. It packages the app with all its dependencies so that it runs the same everywhere.

→ Container vs VM

VM (Virtual Machine)	Container (Docker)
• Each VM has its own OS • Heavy weight • Takes GBs of space • Slower to start • Less efficient	• Shares host OS kernel • Light weight • Takes MBs of space • Starts in seconds • More efficient

→ Docker Architecture

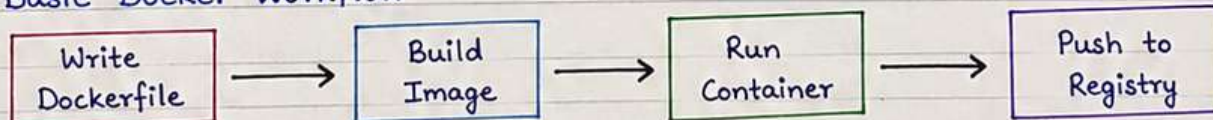


- Client sends commands to Docker Daemon.
- Daemon manages Images, Containers and communicates with Registry.

→ Key Concepts

- **Image**: Read-only template with instructions to create a container.
- **Container**: Running instance of an image.
- **Dockerfile**: Blueprint to build an image.
- **Registry**: Remote storage for Docker images (e.g. Docker Hub).

→ Basic Docker Workflow



→ Important Basic Commands

<code>docker pull <image></code>	# pull image from registry
<code>docker run <image></code>	# run container from image
<code>docker ps</code>	# list running containers
<code>docker ps -a</code>	# list all containers
<code>docker stop <container_id></code>	# stop a container
<code>docker rm <container_id></code>	# remove a container
<code>docker images</code>	# list images

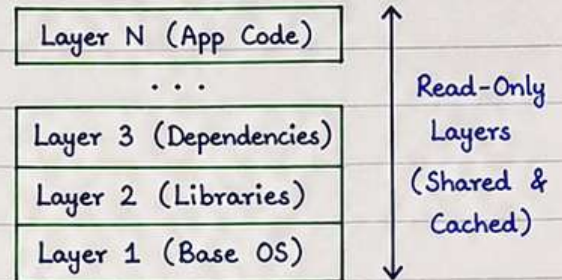
CRASH COURSE

2. DOCKER IMAGES & DOCKERFILE

→ What is a Docker Image?

A Docker Image is a lightweight, standalone, executable package that includes everything needed to run a piece of software: code, runtime, libraries, tools, etc.

Image Layers (Union File System)



→ Dockerfile

A Dockerfile is a text file that contains instructions to build a Docker Image.

Example Dockerfile

Common Dockerfile Instructions

INSTRUCTION	DESCRIPTION
FROM	Base image to start with.
WORKDIR	Set working directory inside the container.
COPY	Copy files/folders from host to image.
ADD	Copy files and also supports URL & auto-extract.
RUN	Execute command while building the image.
EXPOSE	Inform Docker that container listens on the given port.
ENV	Set environment variables.
CMD	Default command to run container.
ENTRYPOINT	Configure a container that will run as an executable.

```
# Base Image
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy package.json
COPY package*.json ./

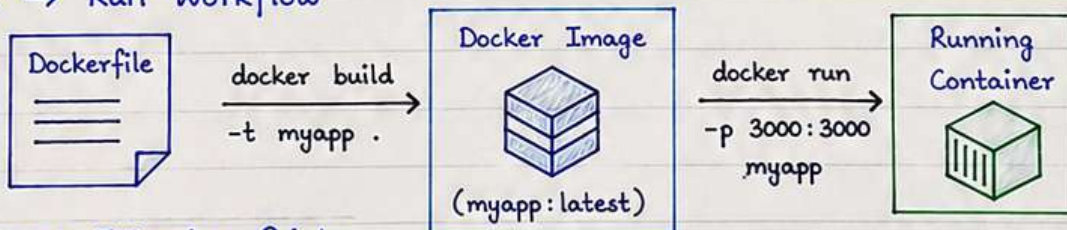
# Install dependencies
RUN npm install

# Copy app source
COPY . .

# Expose port
EXPOSE 3000

# Default command
CMD ["node", "index.js"]
```

→ Build → Run Workflow



→ Important Interview Points

- Images are immutable.
- Containers are created from images.
- Each instruction in Dockerfile creates a new layer (cached for faster builds).
- Keep Docker Image small → use .dockerignore, multi-stage builds.
- Prefer Alpine based images for minimal size.

Tip 💡

Small images = Fast builds + Less attack surface.

Useful Image Commands

- | | |
|--|---|
| • <code>docker images</code> # list images | • <code>docker inspect <image></code> # details |
| • <code>docker rmi <image_id></code> # remove image | • <code>docker pull <image></code> # pull image |
| • <code>docker tag <name:tag></code> # tag image | • <code>docker push <image></code> # push to registry |
| • <code>docker history <image></code> # image layers | |

CRASH COURSE

3. DOCKER NETWORKING & STORAGE

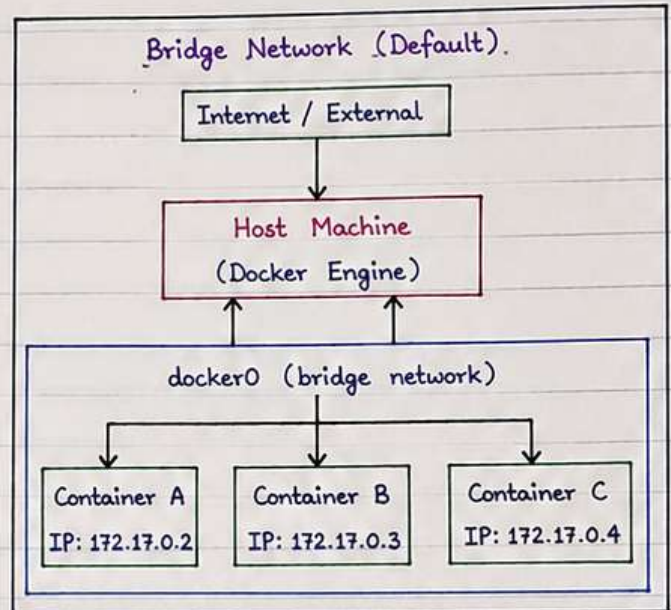
→ Containers need to communicate with each other and with outside world.

CONTAINER NETWORKING

By default, Docker creates a Network named "bridge".

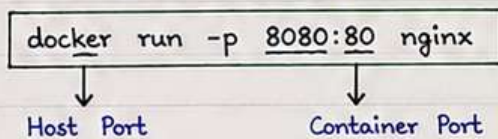
Types of Networks:

- ① Bridge (default) - Containers can communicate with each other on same host.
- ② Host - Container shares the host network.
- ③ None - No networking.
- ④ Overlay - For multi-host communication (used in Docker Swarm / K8s).



PORT MAPPING

Containers are isolated. To access an app running in container from outside, we map container port to host port.



Note:
-p <host_port>:<container_port>

STORAGE IN DOCKER

Containers are ephemeral (data will be lost if container is removed). To persist data we use Volumes or Bind Mounts.

VOLUMES	BIND MOUNTS
- Managed by Docker - Stored in Docker host filesystem (/var/lib/docker/volumes/) - Better for database, production - Independent of container lifecycle	- Map a host directory/file to container - Changes reflect immediately - Good for development - Depends on host path

USEFUL COMMANDS

```

docker network ls           # list networks
docker network inspect <network_name>  # inspect network
docker run --network <network_name> <image>  # run container on network
docker volume ls           # list volumes
docker volume inspect <volume_name>        # inspect volume
docker run -v <volume_name>:/data <image>   # use volume
docker run -v /host/path:/app <image>      # bind mount
    
```

Example

```

docker run -d -p 5000:5000 \
  --name webapp \
  --network mynet \
  -v mydata:/data \
  nginx
    
```

KEY TAKEAWAYS

- ✓ Containers are isolated by default.
- ✓ Use bridge network for communication on same host.
- ✓ Map ports to expose apps to outside world.
- ✓ Use volumes/bind mounts for persistent data.

★ Remember:

Container is temporary,
Data should be permanent!



CRASH COURSE

4. DOCKER COMPOSE

→ Why Docker Compose ?

Running multiple containers with many commands is difficult.

Docker Compose helps us define and run multi-container applications using a single YAML file.

Benefits

- ★ Define multi-container apps.
- ★ Start / Stop all services together.
- ★ Manage Networks & Volumes easily.
- ★ Easy to share and version control.

→ docker-compose.yml (Structure)

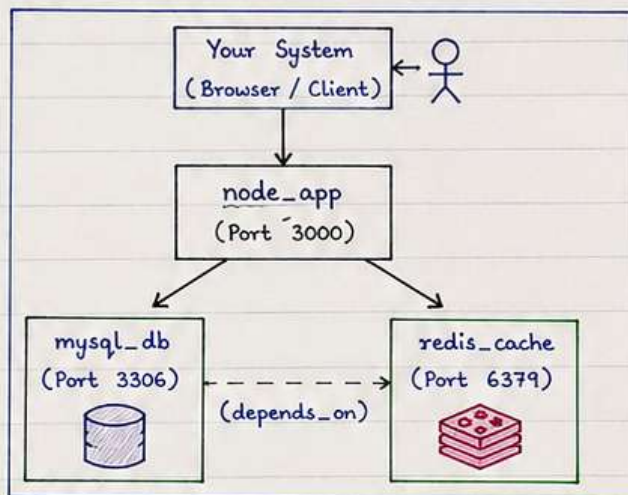
```
version:    version of compose file
services:  all services (containers) of our application
  image:    docker image to use
  container_name: name of the container
  ports:    port mapping
  environment: environment variables
  volumes:  volume or bind mounts
  depends_on: service dependencies
```

→ Example: Node.js + MySQL + Redis

```
version: '3.8'
services:
  app:
    build: ./app
    container_name: node_app
    ports:
      - "3000:3000"
    environment:
      - DB_HOST=db
      - REDIS_HOST=redis
    depends_on:
      - db
      - redis
  db:
    image: mysql:8
    container_name: mysql_db
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: myapp
    volumes:
      - db_data:/var/lib/mysql
    ports:
      - "3306:3306"
  redis:
    image: redis:7
    container_name: redis_cache
    ports:
      - "6379:6379"

volumes:
  db_data:
```

→ Architecture



→ Useful Commands

- docker compose up - start all services
- docker compose up -d - start in detached mode
- docker compose down - stop & remove containers
- docker compose ps - List running services
- docker compose logs -f - view logs
- docker compose build - build / rebuild images
- docker compose down -v - remove volumes also.

Tip



Use .env file to store environment variables and keep yaml clean.

CRASH COURSE

5. KUBERNETES FUNDAMENTALS

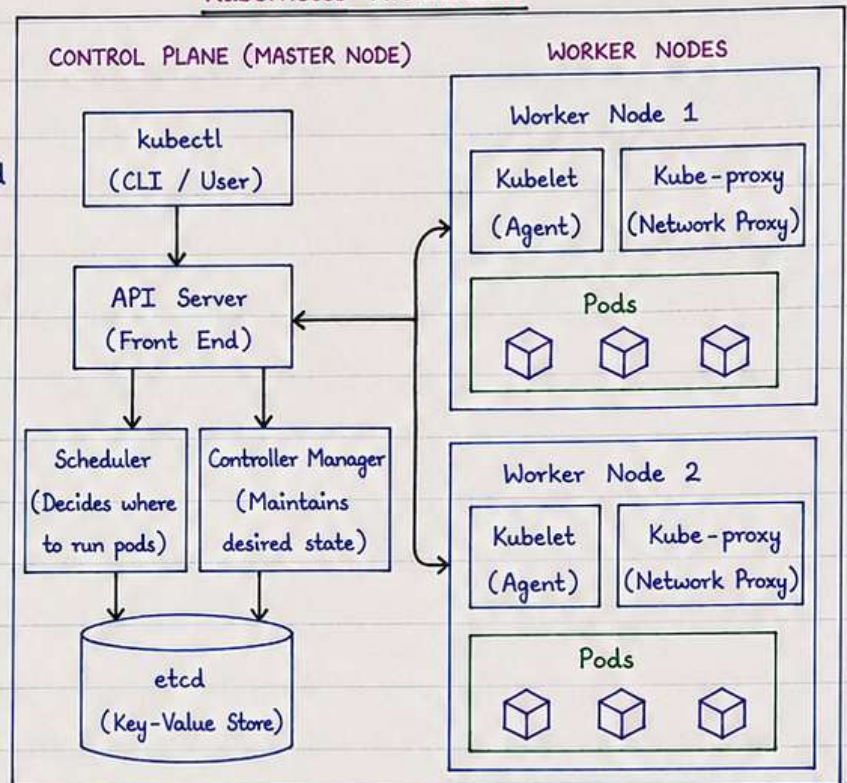
→ What is Kubernetes ?

Kubernetes (K8s) is an open-source platform to automate deployment, scaling, and management of containerized applications.

→ Why Kubernetes ?

- Automated deployments & rollbacks
- Horizontal scaling
- Self-healing
- Service discovery & load balancing
- Secret & configuration management

Kubernetes Architecture



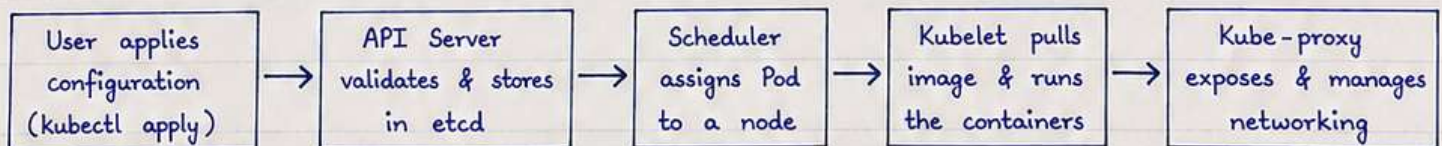
→ Control Plane Components

- **API Server** : Entry point for all K8s operations.
- **etcd** : Consistent & highly available key-value store.
- **Scheduler** : Schedules Pods on worker nodes.
- **Controller Manager** : Runs controllers (Node, ReplicaSet, etc.)

→ Worker Node Components

- **Kubelet** : Agent that ensures containers are running in a Pod.
- **Kube-proxy** : Handles networking and service routing.
- **Container Runtime** : Runs containers (containerd, Docker etc.)

→ How Kubernetes Works ? (High Level Flow).



Key Points

- ✓ K8s = Container Orchestration Platform.
- ✓ Control Plane manages the cluster.
- ✓ Worker Nodes run the application (Pods).
- ✓ Everything is API-driven.
- ✓ Kubernetes is extensible & cloud-agnostic.

Tip

Think of Kubernetes as the brain that manages a cluster of machines and makes sure the right containers are running at all times.



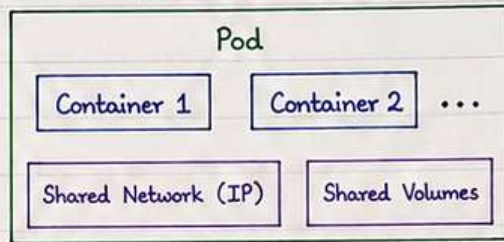
CRASH COURSE

6. KUBERNETES CORE OBJECTS

→ ① POD

- Smallest deployable unit in K8s.
- Represents one or more containers that share the same network & storage.
- Pods are ephemeral (short-lived).
- Each Pod gets a unique IP.

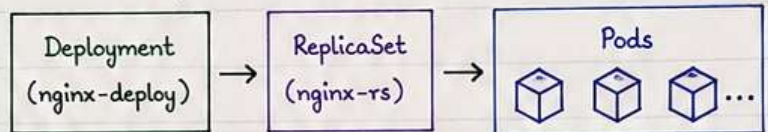
Pod Architecture



→ ② DEPLOYMENT

- Manages ReplicaSets and provides declarative updates for Pods.
- Supports rolling updates & rollbacks.

Deployment → ReplicaSet → Pods



→ ③ REPLICASET

- Ensures that a specified number of Pod replicas are running at any time.
- If a Pod fails, it creates a new one.

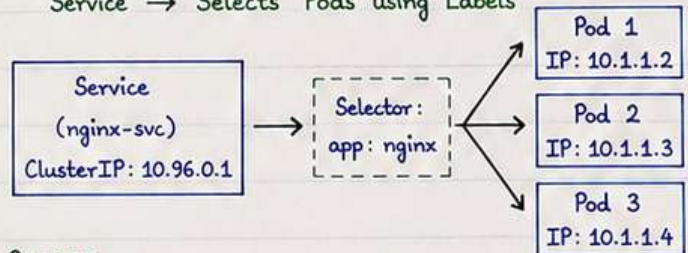
Key Relationship

Deployment manages ReplicaSet.
ReplicaSet ensures desired number of Pods.

→ ④ SERVICE

- Provides a stable IP and DNS name to access Pods.
- Handles load balancing & service discovery.

Service → Selects Pods using Labels



→ ⑤ CONFIGMAP

- Stores non-sensitive configuration data in key-value pairs.
- Used as environment variables, files or command-line arguments.

→ ⑥ SECRET

- Stores sensitive data like passwords, tokens, keys, etc.
- Data is Base64 encoded (not encrypted).
- Used as env vars or mounted as files.

→ ⑦ NAMESPACE

- Virtual cluster inside a physical cluster.
- Helps in resource isolation & organization.
- Default namespaces: default, kube-system, kube-public.

Quick Summary

OBJECT	PURPOSE
Pod	Runs one or more containers together.
Deployment	Manages ReplicaSets & rolling updates.
ReplicaSet	Ensures desired number of Pod replicas.
Service	Provides stable access to Pods.
ConfigMap	Stores non-sensitive configuration.
Secret	Stores sensitive data securely.
Namespace	Provides isolation & organization.

Example: Deployment (nginx)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
    
```



Remember:

Kubernetes = Automate • Scale • Heal • Secure



CRASH COURSE

7. DEPLOYMENT & OPERATIONS IN KUBERNETES

ROLLING DEPLOYMENT

- Update application without downtime.
- New version is rolled out to some Pods, then to others gradually.
- If something goes wrong, we can rollback to previous version.

SCALING

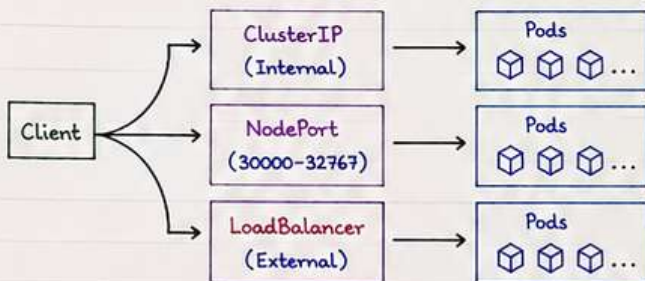
- Scale up / down Pods based on load.
- Manual: `kubectl scale`
- Auto: HPA (Horizontal Pod Autoscaler)

SELF-HEALING

- K8s continuously monitors Pods.
- If a Pod fails, it is restarted automatically.
- Ensures desired state is always maintained.

SERVICE TYPES

- ① ClusterIP - Internal access (default).
- ② NodePort - Access via `<NodeIP:Port>`.
- ③ LoadBalancer - Exposes service externally using cloud LB.



PROBES (HEALTH CHECKS)

- Liveness Probe → Is container running? If fails, container is restarted.
- Readiness Probe → Is container ready to serve traffic?

Example (Liveness Probs)

```

livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
  failureThreshold: 3
    
```

ESSENTIAL KUBECTL COMMANDS

COMMAND	PURPOSE
<code>kubectl get pods</code>	List all Pods
<code>kubectl get all</code>	List all resources
<code>kubectl describe <resource></code>	Details of a resource
<code>kubectl logs <pod-name></code>	View logs of a Pod
<code>kubectl exec -it <pod> -- /bin/bash</code>	Access shell in a Pod
<code>kubectl apply -f <file.yaml></code>	Create / Update resources
<code>kubectl delete -f <file.yaml></code>	Delete resources
<code>kubectl scale deployment <name> --replicas=3</code>	Scale deployment
<code>kubectl rollout status deployment/<name></code>	Check rollout status
<code>kubectl rollout undo deployment/<name></code>	Rollback deployment

DOCKER VS KUBERNETES

DOCKER	KUBERNETES
Containerization platform	Container orchestration platform
Build, Ship & Run containers	Deploy, Scale & Manage containers
Single host	Cluster of machines
Manual scaling	Auto scaling
No self-healing	Self-healing
No scheduling	Smart scheduling
Docker Compose for multi container apps	K8s for large scale, complex applications

TOP 10 DOCKER & K8s INTERVIEW QUESTIONS

- ① What is Docker and how does it work?
- ② What is the difference between Image and Container?
- ③ What is Dockerfile? Explain important instructions.
- ④ What is Docker Compose and why do we need it?
- ⑤ What is Kubernetes and why is it used?
- ⑥ Explain Kubernetes architecture.
- ⑦ What is a Pod in Kubernetes?
- ⑧ What is the difference between Deployment and ReplicaSet?
- ⑨ What are ConfigMap and Secret in Kubernetes?
- ⑩ What is the difference between Liveness and Readiness Probe?

★ QUICK REVISION ★

- ✓ Docker → Build, Ship & Run containers.
- ✓ Kubernetes → Orchestrate, Scale & Manage.
- ✓ Pod → Smallest deployable unit in K8s.
- ✓ Deployment → Manages ReplicaSets.
- ✓ Service → Stable access to Pods.
- ✓ ConfigMap → Configuration as key-value pairs.
- ✓ Secret → Sensitive data securely.
- ✓ K8s ensures → Scalability, Availability, Self-healing & Zero-downtime.

